

```
!python --version !pip show tensorflow
```

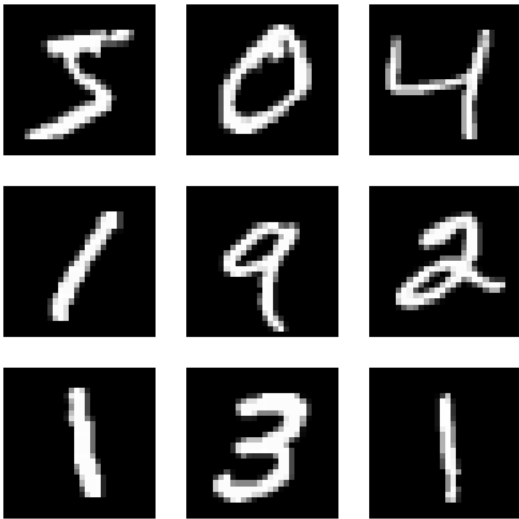
Python 3.12.12

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import numpy as np
import matplotlib.pyplot as plt

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize data (scale pixel values between 0 and 1)
x_train, x_test = x_train / 255.0, x_test / 255.0

# Show sample images
plt.figure(figsize=(5,5))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i], cmap="gray")
    plt.axis('off')
plt.show()
```



```
# Define model architecture
model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(28,28)), # Input layer (flatten 28x28 pixels)
    tf.keras.layers.Dense(128, activation='relu'), # Hidden layer
    tf.keras.layers.Dense(10, activation='softmax') # Output layer (10 classes)
])

# Compile model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train model
model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))

# Save trained model
model.save("mnist_model.h5")
print("Model training complete and saved as mnist_model.h5")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape` to `inp
super().__init__(**kwargs)
```

```
Epoch 1/5
1875/1875 ————— 10s 4ms/step - accuracy: 0.8710 - loss: 0.4412 - val_accuracy: 0.9595 - val_loss: 0.1310
Epoch 2/5
1875/1875 ————— 7s 4ms/step - accuracy: 0.9644 - loss: 0.1191 - val_accuracy: 0.9723 - val_loss: 0.0975
Epoch 3/5
1875/1875 ————— 8s 4ms/step - accuracy: 0.9769 - loss: 0.0768 - val_accuracy: 0.9751 - val_loss: 0.0806
Epoch 4/5
1875/1875 ————— 6s 3ms/step - accuracy: 0.9845 - loss: 0.0517 - val_accuracy: 0.9725 - val_loss: 0.0823
```

Epoch 5/5

1875/1875 ————— 8s 4ms/step - accuracy: 0.9885 - loss: 0.0391 - val_accuracy: 0.9781 - val_loss: 0.0734

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format Model training complete and saved as mnist_model.h5

```
# Load trained model
model = tf.keras.models.load_model("mnist_model.h5")

# Convert to TensorFlow Lite
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

# Save the converted model
with open("mnist_model.tflite", "wb") as f:
    f.write(tflite_model)

print("Model successfully converted and saved as mnist_model.tflite")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until Saved artifact at '/tmp/tmp0ca_derk'. The following endpoints are available:

```
* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28), dtype=tf.float32, name='input_layer')
Output Type:
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)
Captures:
  133426961129616: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133426961128464: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133426961129232: TensorSpec(shape=(), dtype=tf.resource, name=None)
  133426961122128: TensorSpec(shape=(), dtype=tf.resource, name=None)
Model successfully converted and saved as mnist_model.tflite
```

```
# Load TensorFlow Lite model
interpreter = tf.lite.Interpreter(model_path="mnist_model.tflite")
interpreter.allocate_tensors()
```

```
# Get input and output tensor details
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

print("Input Details:", input_details)
print("Output Details:", output_details)
```

Input Details: [{'name': 'serving_default_input_layer:0', 'index': 0, 'shape': array([1, 28, 28], dtype=int32), 'shape_signature': a
Output Details: [{'name': 'StatefulPartitionedCall_1:0', 'index': 9, 'shape': array([1, 10], dtype=int32), 'shape_signature': a
/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning: Warning: tf.lite.Interpreter
TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
See the [migration guide](<https://ai.google.dev/edge/litert/migration>)
for details.

```
warnings.warn(_INTERPRETER_DELETION_WARNING)
```

```
# Select a test image
test_image = x_test[0].astype(np.float32) # Ensure data type matches model input
test_image = np.expand_dims(test_image, axis=0) # Reshape for model input

# Set the input tensor
interpreter.set_tensor(input_details[0]['index'], test_image)

# Run inference
interpreter.invoke()

# Get the prediction
output_data = interpreter.get_tensor(output_details[0]['index'])
predicted_label = np.argmax(output_data)

# Display the image and prediction
plt.imshow(x_test[0], cmap="gray")
plt.title(f"Predicted Label: {predicted_label}, Actual Label: {y_test[0]}")
plt.show()
```

Predicted Label: 7, Actual Label: 7

